The Test the Metaphor Benchmark Never Ran

Vasili Gavrilov*

Draft, August 2026

Abstract

The largest benchmark of metaphor-based optimization heuristics ran 294 implementations, its twelve baselines included, for 1,411,200 runs on the 24 BBOB functions. It contains no statistical test: the word does not occur in it, and none of the 21 papers citing it adds one. Its raw data is public, and the missing test is computable from it. We pair every implementation with random search per function, instance and repetition at the benchmark's own budgets, apply the exact one-sided sign test, and correct with Holm across the 296 implementations its release contains. At the top budget of $10^4 d$ evaluations, 66 to 98 implementations per dimension, one in three in 2D, are not shown better than uniform random sampling at equal cost, and 53 to 79 of those are shown strictly worse. The count of implementations worse than random falls to a mid-budget minimum and rises again toward the top budget: a search that stalls loses ground to a control that keeps sampling. The verdicts survive the benchmark's own anytime measure, a magnitude-aware test, and the choice of control, 121 of 7,104 changing between the benchmark's own random search and a true uniform control. They do not survive a change of implementation: the same algorithm name carries opposite verdicts across libraries in eight of the cases where a name appears twice, and mealpy's two differential evolutions split the same way inside one library. The audit is pre-registered, byte-reproducible, and one program run against the benchmark's own release.

1 A million runs and no test

Vermetten, Doerr, Wang, Kononova and Bäck [1] benchmarked the metaphor-heavy optimization libraries at a scale nobody had attempted: 282 implementations from mealpy 2.5.3, Opytimizer 3.1.2, NiaPy 2.0.5 and the unversioned vendored EvoloPy, plus twelve baselines, the 24 noiseless BBOB functions [6], dimensions 2, 5, 10 and 20, ten instances, five repetitions, $10^4 d$ evaluations each, collected with IOHexperimenter [7]. The paper ranks by an anytime measure, reads the same data at fixed budgets, and counts, per function and descriptively, the implementations random search beats. It draws no statistical conclusion, because it computes no test: the word does not occur in the paper. The direction of the answer is in its own figures; what is missing is the inference.

That is not a criticism of the collection; it is the opening the collection leaves. The polemic against metaphor methods has run for a decade on Sörensen's terms [3], and one floor every implementation can be held to is still unchecked: at a given budget, is this method shown to beat drawing points at random? A floor is not a ranking, and clearing it recommends nothing; failing it at equal cost disqualifies. The benchmark's release [2] carries every run, so the question has an exact answer that requires no new optimization run at all. This paper computes it.

*Independent researcher. The pre-registration, every script, and the audit's C driver: `https://github.com/Anode1/cjitter`; the data is the benchmark's own release, Zenodo 10.5281/zenodo.10561215.

2 The data and the two controls

The release's processed fixed-budget files carry, per implementation, function, instance and repetition, the best error reached by budgets $b \cdot d$ for $b \in \{10, 50, 100, 500, 1000, 5000, 10000\}$. We audit what the release contains: 296 implementations across the four libraries (14 EvoloPy, 53 NiaPy, 82 Opytimizer, 147 mealpy) beside the twelve baselines. The paper's own count is 282; the 14 rows beyond it, eight Opytimizer and six mealpy, all have incomplete panels, and the top-budget headline on the 282 alone is 91, 72, 66 and 61, so nothing turns on them. 278 of the 296 have the full $24 \times 10 \times 5 = 1200$ runs per dimension; the other 18 are audited on the runs present, 19 pairs in the worst case, with coverage printed beside every verdict.

Two controls, because the benchmark's own is not what its name says. The release's `RandomSearch` is nevergrad's: a Gaussian pushed through the bound transform of a `set_bounds` array, concentrated toward the centre of the box. The second control is uniform on the box, drawn by our own library's control method through the COCO C evaluator [6], same functions, instances, dimensions, budget grid and repetition count, deterministic from declared seeds. The two evaluators agree to 7×10^{-10} on 180 probe points, so pairing across them is sound.

3 The test

Within one (implementation, dimension, budget), every run is paired with the control's run on the same function, instance and repetition index, 1200 pairs when coverage is full. The panel weighs the 24 functions equally, so the estimand is performance on that mixture, and at full coverage better means winning at least 663 of the 1200 pairs, a 55.2 per cent win share; Section 4 reports how unevenly that mixture is composed. A strict error win counts for the implementation, a strict loss against it, a tie for neither. Under the null that the implementation is no better than the control, wins are at most a fair coin, and the exact one-sided binomial tail $p = 2^{-n} \sum_{k \geq w} \binom{n}{k}$ is the sign test. We test both directions, correct with Holm [5] across the 296 implementations within each (dimension, budget), separately per direction, and declare at 5% after correction: *better*, *worse*, or *not shown*, where not shown is a failure to demonstrate and is never read as equality. Two directions at 5% each hold each directional family at 5% and the pair at no worse than 10%; at 2.5% per direction, which holds the pair at 5%, 27 of the 7,104 verdicts move and the top-budget row of the table below becomes 197, 216, 224 and 230 shown better. The pre-registered 5% tables are primary and both rules ship in the verdict files. The exact tail also assumes independent pairs, and the release does not fully grant that: every library seeds numpy once per repetition, so pairs share randomness across functions, and the baselines share the control's seeds outright; the robustness paragraph of Section 4 re-tests at the (function, instance) level, where the sharing cannot reach. The budget factor 10 is excluded by pre-registration: below most population sizes the compared value is the shared-seed initial sample, and one baseline has no data there. The design, families, tie rule and exclusions were frozen in a pre-registration before any confirmatory number was computed, with the exploratory pilot it followed disclosed inside it. The pilot had already seen the counts against the benchmark's own random search, so the uniform-control column is the confirmatory one, and the freeze is attested by nothing stronger than this repository's history.

4 What the audit finds

Against the uniform control, per dimension at the top budget of $10^4 d$ evaluations:

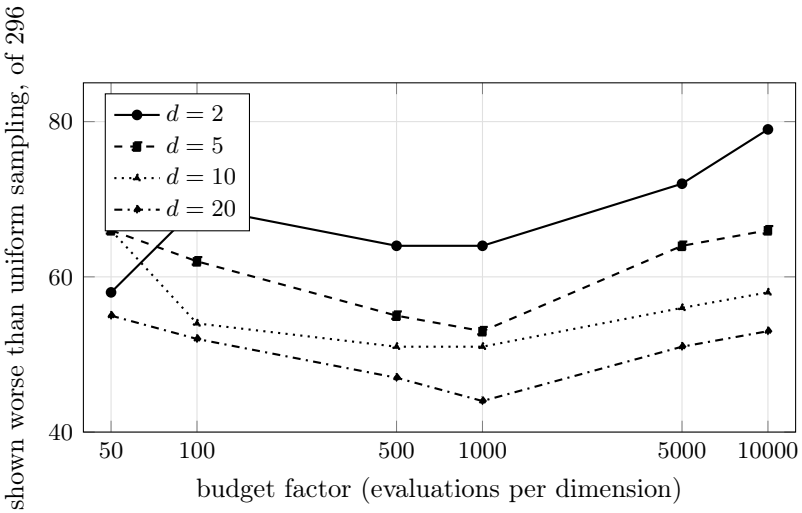

Figure 1: Implementations shown strictly worse than uniform sampling at equal budget, of 296, exact sign test with Holm at 5% per (dimension, budget). The count rises toward the top budget in every dimension, from a mid-grid minimum in 5, 10 and 20 dimensions: a search that stalls is overtaken by a control that keeps sampling.

	$d = 2$	$d = 5$	$d = 10$	$d = 20$
shown better than uniform sampling	198	217	224	230
not shown	19	13	14	13
shown worse	79	66	58	53
not shown better, total	98	79	72	66

One implementation in three in 2D, one in four or five in the higher dimensions, spends $10^4 d$ evaluations without demonstrated advantage over drawing points uniformly at random; the clear majority of those are strictly worse. Against the benchmark's own random search the row reads 100, 79, 71, 65: the audit's verdicts are not an artifact of the control's distribution, since only 121 of the 7,104 verdict cells change between the two controls, most in 2D at the two smallest budgets.

Worse grows with budget. Figure 1 has the counts. In 2D they run 58, 69, 64, 64, 72, 79 across the six budget factors; in 5, 10 and 20 dimensions the minimum sits in the middle of the grid, at $500d$ to $1000d$ evaluations, and only in 2D does the top-budget count exceed the smallest-budget one. The mechanism is not subtle: uniform sampling improves for as long as it runs, and an implementation that stalls, on a bound, on a collapsed population, on its own attraction to the centre, is overtaken and then left behind. A method can be honestly better than random at $500d$ evaluations and honestly worse at $10^4 d$.

The implementation, not the metaphor. Eleven algorithm names appear, as exact strings, in two or more libraries. At the top budget the verdict disagrees across libraries for eight (name, dimension) cells: mealpy's `DE` is shown worse than uniform sampling in all four dimensions while EvoloPy's and Opytimizer's DE are shown better, and SCA splits the same way between Opytimizer (worse) and EvoloPy (better). The split does not even need two libraries: mealpy ships `DE` and `BaseDE`, the first shown worse in every dimension, the second shown better in every dimension. Differential evolution is not a metaphor method, which is the point: the audited unit is a shipped implementation, and a verdict attaches to code, not to the idea the code is named after.

The verdicts are not an artifact of the analysis. Re-run at the (function, instance) level, the sign test on up to 240 median differences per implementation, the top-budget worse counts are 73, 63, 57 and 49 against the table's 79, 66, 58 and 53. On the benchmark's own anytime measure, per-run AOCC against its own random search, the not-shown-better row is 96, 79, 74 and 67. A magnitude-aware paired Wilcoxon gives 90, 69, 59 and 50 worse. Per function the mixture is uneven: at the 10D top budget the shown-worse count runs from 18 on f19 to 85 on f11, the function on which the original paper had noticed random search winning; the pooled verdict averages over exactly this spread. Every one of these re-analyses is one block of the shipped robustness script.

Effect sizes. Beside each verdict we report the median over pairs of the error ratio implementation over control. At the top budget the median implementation delivers 5 per cent of uniform sampling's error in 2D and 22 to 27 per cent in the higher dimensions; 71 implementations reach the 2D optimum to machine precision, ratio exactly zero; 64 to 87 per dimension sit at ratio 1 or above, at or behind the control; the worst sits at 3.8×10^3 times the control's error in 2D. Clearing the floor by an order of magnitude is the common case, and no verdict column shows that, which is why the ratios ship beside the verdicts.

Checks. The audit's own controls behaved: differential evolution, the two modular CMA variants, L-SHADE and BIPOP are shown better in every dimension at the top budget, with win rates from 963 to 1200 of 1200 against their random search and from 983 to 1200 against uniform. One baseline anomaly is pre-registered for verification against the raw logs and reported in Section 5.

5 What this does and does not say

BBOB is shifted and rotated. Kudela [4] showed that several metaphor methods, grey wolf and whale among them, concentrate their search at the centre of the box and collapse when the optimum moves. BBOB moves it by construction. So a worse-than-random verdict here is a verdict about behaviour on shifted problems, a published failure mode, and the audit's claim is exactly "not shown better than random sampling on this benchmark at these budgets", never "worthless everywhere". The near-identity of the two controls' verdicts adds one fact to that discussion: the centre-concentration of the benchmark's own baseline moved almost nothing.

The nevergrad PSO anomaly. The release's own `ng_PSO` baseline is shown worse than random in 10 and 20 dimensions at every budget, 35 wins of 1200 at the 10D top budget. As pre-registered, the row was re-derived from the raw IOHprofiler logs in the release before being stated: 105 fixed-budget values across three cells, differential evolution's among them, match the processed data to the digit, and the anomaly is the implementation's own. On the 10D sphere its five runs end at errors of 18.6 to 34.6 after 10^5 evaluations. The audit reads baselines and metaphors with the same test, and this row is the demonstration.

Coverage. Eighteen implementations have incomplete panels in the release, consistent with its Readme's warning about split raw folders; they are audited on the pairs present and marked. A sensitivity pass restricted to complete 1200-pair cells changes exactly one verdict anywhere in the confirmatory grid, a borderline not shown that becomes better when the Holm family shrinks (mealpy's ImprovedSFO, 5D, budget $5000d$, corrected p crossing at 0.049), and no cell of the top-budget table; every other count falls exactly by the excluded rows' own verdicts.

The code has moved. The release pins mealpy 2.5.3, NiaPy 2.0.5, Opytimizer 3.1.2 and nevergrad 1.0.0; mealpy alone is two major versions on since. Every verdict here attaches to those frozen versions, which is this paper's own thesis applied to itself.

One benchmark, one budget grid. Nothing here transfers to constrained, discrete or expensive problems, and a method not shown better at $10^4 d$ may be the right tool at $10d$: the

budget trend above is the demonstration.

6 The instrument

The tests are two public functions of a small C library, `cjitter_sign_p` and `cjitter_holm`, the same two its comparison harness uses to judge its own four search methods against a matched-budget uniform control; the audit's driver reads the release's runs and prints every verdict in this paper in 18 seconds; the full verdict tables under both controls ship beside this paper in `audit-data/`, with the robustness re-analyses one script beside the driver. The library exists because the comparison discipline used here, a control at equal budget, paired seeds, an exact test, a declared family, is cheap enough to live inside an API rather than in a paper's methods section.

Auditing at this scale repaired the instrument once: the exact binomial tail, summed as plain doubles, overflows $\binom{n}{k}$ past $n = 1028$ and returned NaN on the first 1200-pair panel. The shipped sum now carries an exact power-of-two scale, changes no byte of any smaller panel, and is pinned against exact rational references. An audit tool that had never met a pooled panel would have printed nothing wrong and nothing at all; the failure surfaced because the tests pin values, which is the discipline this paper is asking benchmarks to adopt.

References

[1] D. Vermetten, C. Doerr, H. Wang, A. V. Kononova, T. Bäck. Large-Scale Benchmarking of Metaphor-Based Optimization Heuristics. In *GECCO 2024*, 41–49. DOI 10.1145/3638529.3654122.

[2] D. Vermetten et al. Reproducibility files for [1]. Zenodo, DOI 10.5281/zenodo.10561215, CC-BY 4.0.

[3] K. Sörensen. Metaheuristics—the Metaphor Exposed. *International Transactions in Operational Research* 22(1):3–18, 2015.

[4] J. Kudela. The Evolutionary Computation Methods No One Should Use. arXiv:2301.01984, 2023.

[5] S. Holm. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6(2):65–70, 1979.

[6] N. Hansen, A. Auger, R. Ros, O. Mersmann, T. Tušar, D. Brockhoff. COCO: A Platform for Comparing Continuous Optimizers in a Black-Box Setting. *Optimization Methods and Software* 36(1):114–144, 2021.

[7] J. de Nobel, F. Ye, D. Vermetten, H. Wang, C. Doerr, T. Bäck. IOHexperimenter: Benchmarking Platform for Iterative Optimization Heuristics. *Evolutionary Computation* 32:205–210, 2024.